START HERE

How to present the Predictive BMS on your Windows PC

Install, open, click, and what to say: the demonstration, the KiCad circuits, the Octave model check and the Scilab Xcos models

Predictive Thermal Battery Management System for Li-ion Battery Packs

Muskan Sulathana · Guide: Dr. M. Rupesh

You do not need to write or change any program. Every part opens with a double-click or a menu, and runs with one button. Read this guide once with the programs open, and practise the 15-minute plan in section 8 two times before the review.

Contents

1. What to install
2. Get the project folder ready
3. Part 1: the demonstration dashboard
4. Part 2: the circuits in KiCad
5. Part 3: the model check in GNU Octave
6. Part 4: the block diagrams in Scilab Xcos
7. Part 5: the Build Guide
8. The 15-minute presentation
9. The day before: checklist
10. If something goes wrong

The project in two sentences

This project protects a battery pack of four lithium-ion cells. An ESP32 controller measures every cell five times a second, uses a heat model to find heat that should not be there, and disconnects the pack early, long before the cells reach a fixed temperature limit.

The hardware is not built yet. Everything you show is a **simulation**: on the computer, with the real circuit values and the real controller program. Always say "in the simulation" when you give a result.

1. What to install

All the programs are free. Install them the day before, when you have good internet. They are large downloads. During each installation, keep the default choices and click **Next** until it finishes.

Program	Where to get it	What it is for
KiCad 10	**kicad.org** → Download → Windows. Take the newest 10.0 version.	The circuit diagrams, and their simulation (KiCad has a circuit simulator inside, called ngspice).
GNU Octave 10.3.0	**octave.org** → Download → Windows. Take the installer for version 10.3.0 (its file name ends with `-w64-installer.exe`).	The heat model and its estimator, written as MATLAB-style files. Octave is the free program that runs MATLAB files.
Scilab 2026.0.1	**scilab.org** → Download → Windows 64-bit installer for 2026.0.1.	Xcos, the block-diagram tool inside Scilab. It is the free tool like Simulink.
Nothing	—	The demonstration dashboard. It runs from the project folder and opens in your normal web browser (Edge or Chrome).

> **First start of KiCad.** The first time KiCad opens, it may ask how to set up the symbol and footprint libraries. Choose the first, default option (*Copy default global ... library table*) and click OK. This happens only once.

2. Get the project folder ready

You get the project as one zip file, `PredictiveBMS-Windows.zip`. If you need a fresh copy, it is on the project's GitHub page, **github.com/KiranPranay/predictive_bms**, under **Releases**.

1. Right-click the zip file → **Properties**. At the bottom, tick **Unblock** (if you see it) → **OK**. This stops Windows from blocking the demonstration later.
2. Right-click the zip file → **Extract All…** Choose a short place, for example `C:\PredictiveBMS` or your Desktop. Do not run anything from inside the zip: extract it first.
3. Open the extracted folder **PredictiveBMS**. You see:

In the folder	What it is
START HERE.pdf	This guide.
1 Demo - double-click.bat	Starts the demonstration dashboard (Part 1).
2 KiCad circuits	Four KiCad projects: cell-taps, ntc-divider, current-sense, thermal-model (Part 2).
3 Octave model check	The Octave files, such as thermal_step.m and crosscheck.m, and the simulator program bms_sim.exe (Part 3).
4 Scilab Xcos models	cell_thermal_model.zcos and abnormal_heat_detection.zcos (Part 4).
Documents	Build-Guide.pdf, Viva-Notes.pdf and this guide.
program	Files the demonstration needs. Do not move or change them.

> If Windows shows a blue box, **Windows protected your PC**, when you start something from this folder: click **More info**, then **Run anyway**. The files are the project's own programs. Unblocking the zip in step 1 avoids most of these boxes.

3. Part 1: the demonstration dashboard

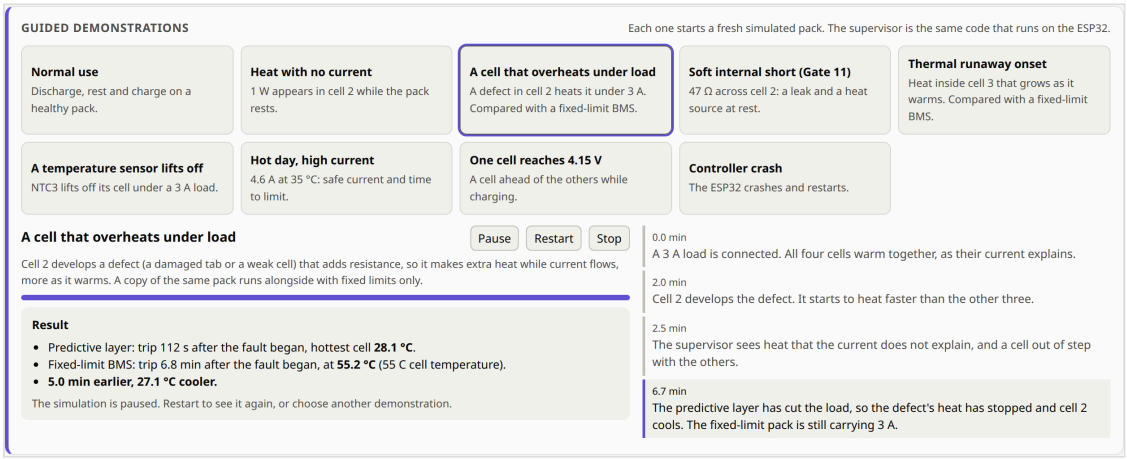
The dashboard shows the whole system working on a **simulated pack**: four cells, their heat, the hardware BMS, the sensors, and the same controller program that will run on the ESP32. A second copy of the pack runs beside it, protected only by fixed limits, like a normal BMS. So every fault shows how much earlier this project acts.

Start it

1. Double-click **1 Demo - double-click.bat**. A black window opens. It says that the demo is running. **Keep this black window open** for the whole presentation. Closing it stops the demo.
2. Your browser opens the dashboard by itself. If it does not, open Edge or Chrome and type `http://127.0.0.1:8080/` in the address bar.
3. Wait a few seconds until the numbers on the page start to change.

What is on the page

- **Guided demonstrations**: nine buttons. Each button starts one story on a fresh pack. A text box tells what is happening, step by step. It has **Pause**, **Restart** and **Stop** buttons.
- **Predictive layer against a fixed-limit BMS**: the comparison of the two packs.
- **Cell cards**: for each cell, its voltage, temperature, estimated core temperature, abnormal heat (in watts) and its risk level.
- **System**: a live drawing of the pack, the relays and the sensors.
- **Charts**: cell temperatures, abnormal heat, and each 5-minute forecast drawn against the real reading.
- **Verification**: all 30 test scenarios with their checks, at the bottom.



The dashboard during a guided demonstration (here: a cell that overheats under load).

Show the demonstrations in this order

Click one button, let it run to the end, read the result box, then click the next. Times are approximate and change by a few seconds between runs.

Order	Button	What happens (about how long)	What it proves
1	**Normal use**	A 2 A load, a rest, then charging at 1.5 A (about 2 minutes). The level stays **Normal**. The 5-minute forecasts land on the real readings.	No false alarms. The model knows how a healthy cell heats.
2	**Heat with no current**	1 W of heat starts inside cell 2 while no current flows (under 1 minute). The supervisor trips about 1 minute later, with the cell near 26 °C. The fixed-limit BMS does nothing.	It finds heat that the current cannot explain, which is how an internal fault starts.
3	**A cell that overheats under load**	Cell 2 gets a defect under a 3 A load (under 1 minute). This project trips about 2 minutes after the fault, near 28 °C, and the cell cools. The fixed-limit BMS trips about 7 minutes after the fault, at 55 °C.	The main result: about 5 minutes earlier and about 27 °C cooler.
4	**Soft internal short (Gate 11)**	A 47 Ω resistor across cell 2 at rest, like a small internal short (about 1 minute). The leak estimate finds about 73 mA, close to the 79 mA really drawn. Heat and leak agree. Level: **Warning**.	Two independent kinds of evidence, electrical and thermal, point to the same cell.
5	**Thermal runaway onset**	Heat inside cell 3 that grows as the cell warms (about 1 minute). This project trips about 8 minutes after it starts, near 29 °C. The fixed-limit BMS trips almost 4 minutes later, at 55 °C.	An early warning. Disconnecting cannot stop heat made inside a cell, so the gain is warning time.

> **Say:** Each demonstration runs on the same controller program that will run on the ESP32. The grey dashed line is the second pack, which has only fixed limits. Look how much earlier this project acts, and how much cooler the cell is.

What to point at

- **Abnormal heat chart**: the line of the faulty cell rises above the others. 0.5 °C/min is Warning and 1.0 °C/min is Trip.
- **Comparison card**: the trip time and the temperature of both packs, side by side.
- **Forecast chart**: for a healthy cell, the forecast made 5 minutes earlier lands on the real reading.
- **Latched trip**: after a trip, both relays stay open until a manual reset is allowed.

To finish, scroll to **Verification** and click **Run again**: **30 of 30 pass**.

> **Say:** These 30 scenarios are the tests of the build gates, done in simulation before any real cell is connected. All 30 pass.

> To stop the demo, close the black window. To start again, double-click the .bat file again.

4. Part 2: the circuits in KiCad

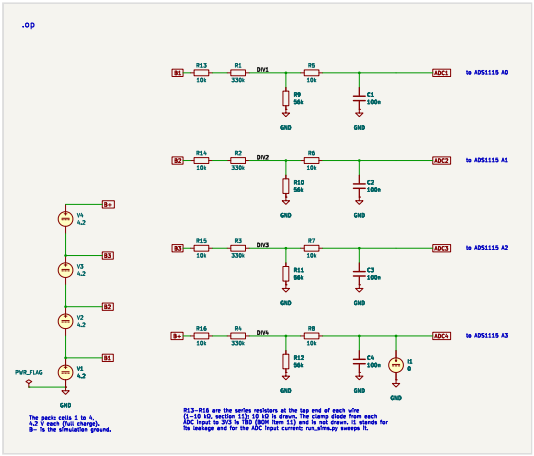
KiCad is the circuit design program used in industry. Here it holds four small circuits, each with its own simulation. The part values are the ones the prototype uses. Each circuit is one folder in **2 KiCad circuits**.

How to open and run any of the four

1. In the folder, open one circuit folder, for example **thermal-model**. Double-click the file ending in `.kicad_pro`. KiCad opens.
2. In the KiCad window, double-click the file ending in `.kicad_sch` (or click **Schematic Editor**). The circuit appears.
3. In the Schematic Editor menu, click **Inspect → Simulator**. The simulator window opens.
4. Click **Run** (the green triangle ▶). If a window asks for the analysis, it is already filled in from the blue command written on the sheet (for example `.tran 1 3610`): click **OK**.
5. Click **Add Signals**, tick the signals named below, and click **OK**. The graph appears.

> **Say:** This is the real circuit with the real part values, drawn in KiCad. KiCad's built-in simulator, ngspice, calculates it. KiCad's electrical rules check finds no errors.

Circuit 1: cell-taps (board A)

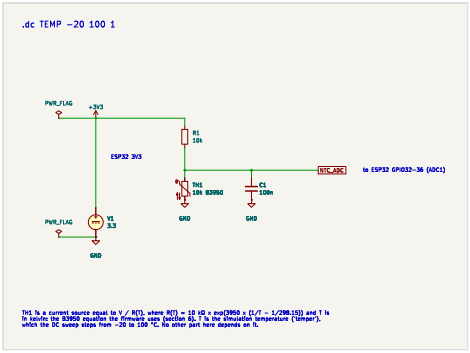


The four cell-tap channels as KiCad draws them.

- **Run:** this sheet uses `.op`, the "operating point": the steady voltages.
- **Read:** the voltages of **ADC1, ADC2, ADC3, ADC4** in the results list: about 0.59, 1.19, 1.78, 2.38 V.
- **Point at:** all four are below 3.0 V, the safe limit for the ADS1115 input (the Gate 1 stop rule).

> **Say:** Each cell tap goes through a 330 kΩ and 56 kΩ divider, so 16.8 V becomes about 2.4 V, which the ADC can read safely. The 10 kΩ and 100 nF filter removes noise.

Circuit 2: ntc-divider (board B)

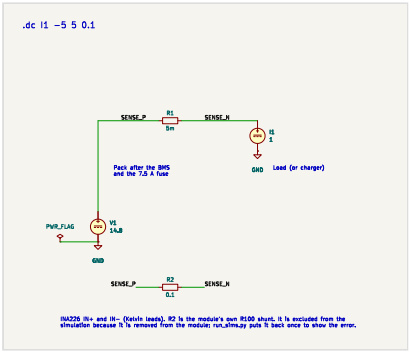


One NTC temperature divider.

- **Run:** this sheet sweeps the temperature of the NTC from −20 to 100 °C (`.dc TEMP -20 100 1`).
- **Add signal: V(NTC_ADC)**. The graph shows the ADC voltage falling as the temperature rises.
- **Point at:** 1.65 V at 25 °C, about 1.00 V at 45 °C (the charge limit) and about 0.76 V at 55 °C (the discharge limit).

> **Say:** An NTC's resistance falls as it gets hotter. With a 10 kΩ resistor above it, the voltage at the ESP32 tells the temperature. It stays in the ESP32's accurate range from about 3 °C to above 100 °C.

Circuit 3: current-sense

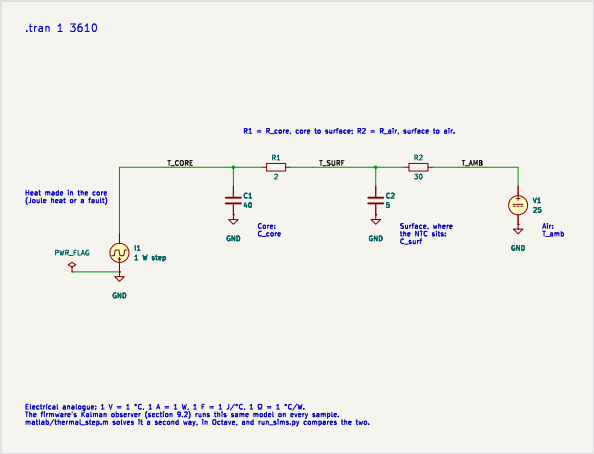


The 5 mΩ shunt and the INA226 inputs.

- **Run:** this sheet sweeps the pack current from −5 A (charging) to +5 A (discharging) (`.dc I1 -5 5 0.1`).
- **Add signal:** the voltage across the shunt, **V(/SENSE_P) − V(/SENSE_N)**. In the simulator, use the **user-defined signal** box (in the signals menu) and type `V(/SENSE_P)-V(/SENSE_N)` . If you cannot find it, add **V(/SENSE_P)** and **V(/SENSE_N)** and explain the small gap between them.
- **Point at:** 25.0 mV at 5 A. The INA226 can read up to about 81.9 mV, so there is a large margin. Discharge reads positive.

> **Say:** A 5 milliohm resistor in the pack's path makes a small voltage proportional to the current. The INA226 chip reads it in both directions, charge and discharge.

Circuit 4: thermal-model



The heat model of one cell, drawn as an electrical circuit.

- **Run:** this sheet simulates one hour after 1 W of heat starts in the cell's core (`.tran 1 3610`).
- **Add signals: V(/T_CORE)** and **V(/T_SURF)**. Here 1 volt means 1 °C.
- **Point at:** both start at 25 °C. After one hour the core is at about 54.5 °C and still rising slowly. It would settle at 57 °C.

> **Say:** Heat flows like current. A heat capacity behaves like a capacitor, and a thermal resistance like a resistor. So the cell's heat model is a simple RC circuit. 1 volt stands for 1 °C and 1 ampere for 1 watt. The ESP32 runs this same model five times a second.

> **Tip.** On the projector, zoom the graph with the mouse wheel, or use the View menu to zoom.

5. Part 3: the model check in GNU Octave

GNU Octave runs MATLAB-style programs (files ending in `.m`). We use it to check the heat model and the estimator a second, independent way. The same files also run in MATLAB.

Open Octave in the right folder

1. Start **GNU Octave (GUI)** from the Start menu.
2. Find the **File Browser** panel (on the left). Use its folder box at the top, or the **Current Directory** box above the Command Window, to go to the folder **3 Octave model check**. The panel now lists `thermal_step.m`, `crosscheck.m` and the other files.

Run 1: thermal_step (a few seconds)

1. Double-click **thermal_step.m**. It opens in the Editor.
2. Press **F5** (or click the Run button ▶ in the Editor). Another way: click in the Command Window, type `thermal_step` and press Enter.
3. A figure opens: the core (orange) and surface (blue) temperatures over one hour after 1 W of heat. The Command Window prints the temperatures at 1, 5, 20 and 60 minutes, and the two time constants, 8.4 s and 1422 s.

> **Say:** This is the same heat model as in KiCad, solved by a different method. KiCad and Octave agree to within 0.0002 °C.

Run 2: crosscheck (one scenario in about a minute; all four take a few minutes)

1. In the Command Window type `crosscheck({{'load_defect'}})` and press Enter: one scenario. (`crosscheck` alone runs all four; run that the day before.)
2. Octave starts `bms_sim.exe`, the simulator, which runs the real controller program and writes a log. Then Octave's own copy of the estimator reads the same log and calculates the same numbers.
3. A figure opens with four panels. Top: the core temperature and the abnormal heat of each cell; the solid (Octave) and dashed (controller) lines lie on top of each other. Bottom: the difference between them: only a few thousandths of a degree and of a watt. (The same figure is in the Build Guide, section 16.5.)

> **Say:** The controller's estimator was written again in Octave and given the same sensor data. Both give the same core temperature and abnormal heat, to a few thousandths. This shows the controller program does what the model says.

6. Part 4: the block diagrams in Scilab Xcos

Xcos is the block-diagram simulator inside Scilab, the free tool like Simulink. Each model is a drawing of blocks joined by arrows. A scope block draws the result.

How to run a model

1. Open the folder **4 Scilab Xcos models** and double-click the model file (`.zcos`). Scilab starts and Xcos opens the diagram. (If it does not: start Scilab, click **Applications** → **Xcos**, then **File** → **Open** and choose the file.)
2. Click **Start**, the black triangle ▶ in the Xcos toolbar.
3. Wait until the simulation ends. The scope windows open with the graphs.

Model 1: cell_thermal_model.zcos

The heat model of one cell as blocks: 1 W of heat starts in the core at 10 s, the air is at 25 °C, and the simulation runs for one hour. The scope shows the core and the surface temperatures. It is the same model as the KiCad thermal-model sheet and Octave's thermal_step.

Point at: [[XCOS-A]]

> **Say:** Here the same heat model is built from blocks, as engineers do in Simulink. The heat goes in on the left, the model is in the middle, and the scope shows the temperatures.

Model 2: abnormal_heat_detection.zcos

One cell carries 3 A. Its normal Joule heat (I^2R) is known. At 10 minutes, 1 W of extra, unknown heat starts: the "abnormal" heat. An observer block, a fixed-gain version of the controller's Kalman filter, watches only the surface temperature and estimates the abnormal heat. Two trips are compared:

- **Predictive trip**: when the estimated abnormal heat reaches 0.75 W (about 1 °C per minute, the architecture's trip level);
- **Fixed-limit trip**: when the surface temperature reaches 55 °C.

The scopes show the temperatures, the estimated against the true abnormal heat, and the two trip signals.

Point at: [[XCOS-B]]

> **Say:** The observer sees only the surface temperature, like the real NTC sensor. It works out how much heat is not explained by the current. The predictive trip comes long before the fixed 55 °C limit.

7. Part 5: the Build Guide

Open **Documents → Build-Guide.pdf**. It is the full construction document. Show these sections:

Section	What to show	What to say
2. The system at a glance	The architecture drawing and the table of the pack	Two layers protect the pack: a hardware BMS, and the ESP32 supervisor with two relays.
3. Single-line diagram	The power path from the cells to the load and the charger	The fuse is at the pack's positive end, then the current shunt, then the two relays.
4. Circuit diagrams	Cell taps, NTCs, current, control	Every value comes from the architecture document.
6. Components	The parts list and the total cost	The full parts list costs about ₹9,260.
8 and 9. Board A and board B	The perfboards hole by hole	The board drawings are checked by a program: every connection is made and nothing is shorted.
12 and 13. Assembly and checks	The step-by-step assembly and the meter checks	Each board is checked with a meter before any power.
14. Bring-up	The gate table	The build goes through Gates 0 to 12. No lithium cell is connected before Gate 3.
15 and 16. The simulation	The demonstration pictures, the KiCad and Octave results	All results so far are simulation results; the gates measure the real circuits.

8. The 15-minute presentation

Before you start: the demo is running (black window open), KiCad has the thermal-model sheet open with its graph ready, Octave shows the thermal_step figure, Xcos has abnormal_heat_detection open, and the Build Guide is open. Then you only switch windows (**Alt + Tab**).

Time	Show	Say
0:00–1:00	Build Guide cover	The title, your name, your guide. "This project predicts heating faults in a lithium-ion pack before a fixed limit is reached."
1:00–2:30	Build Guide section 2 (architecture)	The problem: lithium-ion cells can overheat, and a fixed temperature limit acts late. The two layers: the hardware BMS always protects; the ESP32 adds the predictive layer and two relays.
2:30–3:30	Build Guide sections 3 and 4	The power path and the sensors: four cell voltages (ADS1115), pack current (INA226 with a 5 mΩ shunt), five temperatures (NTCs).
3:30–4:30	Viva Notes, the heat model (or Build Guide 16.4)	The heat model: core and surface. The Kalman filter estimates the core temperature and the abnormal heat in watts.
4:30–9:30	The dashboard: Normal use, Heat with no current, A cell that overheats under load, Soft internal short	One sentence per demonstration (section 3). Point at the comparison card: about 5 minutes earlier and about 27 °C cooler.
9:30–10:30	The dashboard: Verification	30 of 30 test scenarios pass.
10:30–12:00	KiCad: thermal-model graph, then cell-taps values	The circuit values are checked in KiCad's simulator. The heat model is an RC circuit.
12:00–13:00	Octave: the crosscheck figure	The estimator, written again in Octave, gives the same numbers as the controller program.
13:00–14:00	Xcos: abnormal_heat_detection	The block diagram: the predictive trip comes long before the fixed limit.
14:00–15:00	Build Guide section 14 (bring-up)	What is next: build the hardware gate by gate. Safety rules. "Thank you. I am happy to take questions."

> If time is short, skip the soft-short demonstration and the Xcos part. If there is extra time, show the Thermal runaway onset demonstration.

9. The day before: checklist

- ☐ KiCad 10, GNU Octave 10.3.0 and Scilab 2026.0.1 are installed. Each one has been opened once (KiCad's library question is answered).
- ☐ The zip is unblocked and extracted to a short folder, for example `C:\PredictiveBMS` (not inside OneDrive).
- ☐ The demo starts, and all five demonstrations of section 3 run to the end.
- ☐ Each of the four KiCad circuits runs, and you know which signals to add.
- ☐ In Octave, `thermal_step` and the full `crosscheck` have run once.
- ☐ Both Xcos models run and show their scopes.
- ☐ You have read the Viva Notes, and practised the 15-minute plan two times with a timer.
- ☐ Laptop charged; charger and HDMI adapter in the bag; the display set to **Duplicate** (Windows key + P).
- ☐ Sleep turned off (Settings → System → Power) and notifications off (Focus assist / Do not disturb).
- ☐ A copy of the whole PredictiveBMS folder on a pen drive.

Nothing in the project needs the internet during the presentation.

10. If something goes wrong

Problem	What to do
Blue box: **Windows protected your PC**	Click **More info**, then **Run anyway**. Next time, unblock the zip before extracting (section 2).
The black window opens and closes at once, or says a file is missing	Extract the whole zip first; do not run it from inside the zip. Keep the folders as they are. Antivirus may have removed a file: extract again and allow the file.
The browser does not open	Open Edge or Chrome yourself and type `http://127.0.0.1:8080/`.
The black window shows an error about port 8080, or the page does not load	Another demo is already running. Close every black window, wait 5 seconds, and start again. If it still fails, restart the PC.
The page is open but nothing changes	Check that the black window is still open. Press **F5** in the browser to reload.
A demonstration seems stuck	Click **Stop**, then click the demonstration button again.
KiCad asks about the symbol or footprint library table	Choose the first, default option (Copy default global ... table) and click OK.
The KiCad graph is empty after Run	No signal is added yet: click **Add Signals** and tick the signals named in section 4.
KiCad asks for the simulation command	Type the blue command written on the sheet (for example `.tran 1 3610`) and click OK.
Octave says `'thermal_step' undefined`	Octave is in the wrong folder. Change the Current Directory to **3 Octave model check**.
Octave's crosscheck cannot find the simulator (bms_sim.exe)	bms_sim.exe must be in the same folder as crosscheck.m. Extract the zip again; if antivirus removed it, allow it.
Octave's crosscheck is slow	Run only `crosscheck({'load_defect'})`, or show its figure in the Build Guide, section 16.5.
The .zcos file opens in another program	Start Scilab, click **Applications** → **Xcos**, then **File** → **Open**.
The Xcos scope window does not appear	Wait until the simulation ends. Look for the scope behind the Xcos window (use the taskbar). Read the Scilab console for a message, then click Start again.
Text is too small on the projector	In the browser press **Ctrl** and **+**. In KiCad and Xcos use the mouse wheel to zoom.

If one part fails and cannot be fixed quickly, show its picture in the Build Guide (section 15 or 16) and move on. Say: "This is the result from the same simulation."